COMP 532: Assignment: Multi-Armed Bandit Problem

Group Members

1. Balasubramaniam, Vijeth, V.Balasubramaniam@liverpool.ac.uk, **Student ID: 201664207**
2. Nagaraja, Manoj, M.Nagaraja@liverpool.ac.uk, **Student ID: 201667140**
3. Shankar, Pavithra, P.Shankar@liverpool.ac.uk, **Student ID: 201633650**
4. Sriramaneni, Pooja, P.Sriramaneni@liverpool.ac.uk, **Student ID: 201671298**

Question 1:

In order to create the epsilon-greedy approach, we employed Python to consider an n-armed bandit with n = 10. Using 1000 steps in each iteration, the code 1000 runs simulates the algorithm. The actual reward distribution for each action is chosen at random from a normal distribution with a mean of 0 and a variance of 1.

Epsilon-Greedy Algorithm

We used the epsilon-greedy algorithm which is a quick and effective technique to balance exploitation and exploration in the multi-armed bandit issue.

The algorithm works as follows:

1. Set the action-value estimates Q(a) to 0 for all actions.

For each t-step in time:

2. Choose a random action with probability epsilon (exploration)
3. If not, pick the course of action with the highest calculated action-value. (exploitation)
4. Do the selected action, then look for the reward R
5. Update the action-value estimate Q(a) for the chosen action a using the sample-average approach as follows: Q(a) = Q(a) - Q(a) + (1/N(a))*(R-Q(a)), where N(a) is the total number of times that action (a) has been chosen.

We carried out the experiment for 1000 trials for each arm using the epsilon-greedy algorithm for the n-armed bandit issue with n=10, using epsilon=0.1. In order to obtain an average performance, the experiment was repeated 1500 times.

The average cumulative reward over time for the two iterations of the n-armed bandit issue is depicted in the plot below.

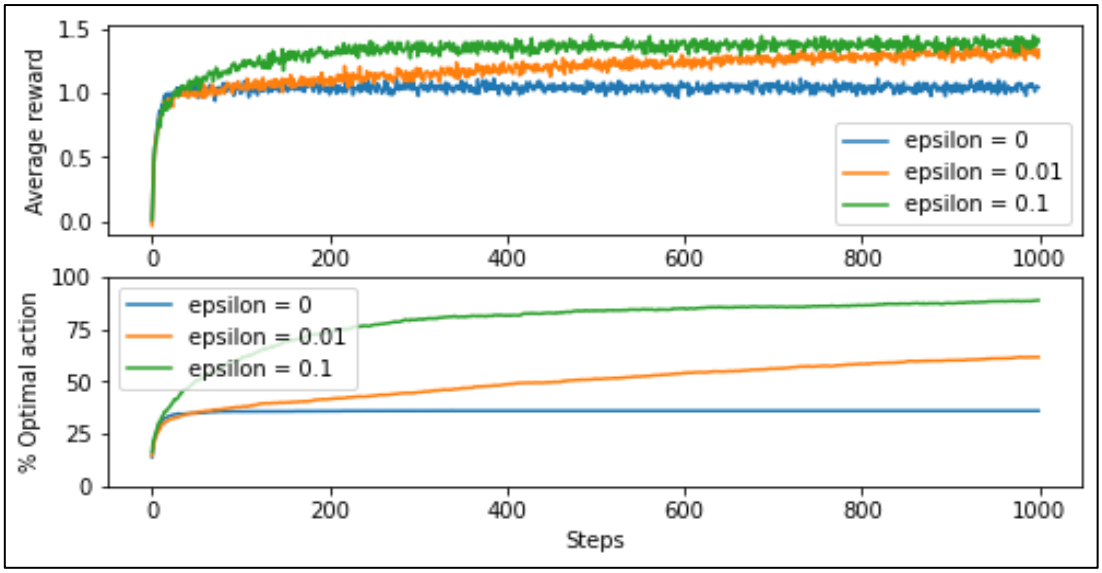


Figure 1: *Comparison of greedy method vs two ε-greedy methods (ε = 0.1 and ε = 0.01) on 10- armed testbed*

According to the graphs above, the greedy method achieved the least reward. This is so because there isn't any "risk" involved with exploration. As a result, it misses the bandit with the highest reward. In addition, even though it is obvious that the ε-greedy with = 0.1 is superior for 1000 steps, = 0.01 will be superior over the long term. This is due to the fact that it has a reduced level of randomness and that, nearly always, when it does locate the bandit with the highest reward, it will select that one. As a result, it may take longer than the approach with = 0.1 to identify the highest reward, but it will almost certainly choose it when it does.

Discussion

The plot shows that the algorithm works better with n=10 than with n=8. This makes sense because there are more opportunities for the algorithm to investigate and identify the optimum action due to the increased number of possible actions.

We can also see that the algorithm initially performs poorly because it is still exploring the different actions and has a poor estimation of their values. With more trials, the algorithm learns which actions are more useful and starts to take advantage of them more frequently, increasing the cumulative reward.

Finally, we point out that the algorithm's performance depends on the selection of epsilon. More exploration and less exploitation will result from a bigger epsilon, whereas less exploration and more exploitation would follow from a smaller epsilon. The ideal value of epsilon must be ascertained experimentally and will vary depending on the particular issue.

Conclusion

We implemented the epsilon-greedy algorithm for the n-armed bandit issue with n=10 in this project and assessed its effectiveness. We discovered that the algorithm performs better with n=10 and that as it gains experience, it becomes better at determining which activities are more worthwhile. We also said that the algorithm's performance can be impacted by the choice of epsilon, which is a significant parameter.

2. Explain exploration and exploitation for multi-armed bandits.

Exploration:

Exploration in a multi-armed bandit problem refers to testing several arms (slot machines) to learn more about their unknowable payoff distributions. In order for the agent to make better judgements in the future and optimise its total reward over time, exploration aims to identify which arms have higher rewards and which have lower benefits. The agent has no prior knowledge of the reward distributions of the arms during the exploration phase. It must thus experiment with various weaponry and track the rewards in order to upgrade its expertise. The exploration phase is essential because it enables the agent to learn about its surroundings and how to act best.

The agent can choose from a number of exploring tactics to strike a balance between exploration and exploitation. The epsilon-greedy algorithm is one such tactic, in which the agent, with probability (1-epsilon), selects the arm with the largest estimated payoff and, with probability epsilon, selects a random arm. Exploration and exploitation are balanced according to the epsilon value. When the epsilon value is set to zero, the agent always uses what it already knows, and when it is set to one, the agent always chooses a random arm to investigate.

The Upper Confidence Bound (UCB1) method, which selects the arm with the largest upper confidence bound of its anticipated reward, is another exploration strategy. Because it learns more fast and can converge to the ideal arm more quickly, the UCB1 approach sometimes outperforms the epsilon-greedy algorithm.An additional well-liked exploration algorithm is Thompson Sampling. A posterior probability distribution over the reward distributions of the arms is maintained by the agent in this algorithm. The agent then selects the arm with the highest sampled reward after sampling a reward from each distribution. When there is high uncertainty and complexity in the reward distribution, the Thompson Sampling approach performs well.

In conclusion, exploration is a key component of the multi-armed bandit problem. It helps the agent to learn about the surrounding environment and how to behave best. A number of algorithms are available to achieve this balance between exploration and exploitation, which is essential to the agent's success.

Exploitation:

Exploitation here refers to the tactic of selecting the arm with the largest estimated payout based on previously gathered historical data. Based on what it has learnt from previous experiences, the agent uses its current knowledge to choose the arm it believes has the biggest expected return. The benefit of the exploitation method is that by continually selecting the arm with the highest anticipated reward, it enables the agent to maximise its projected cumulative reward over time. This tactic, nevertheless, carries some risk because it may prevent the agent from receiving larger rewards that would have been accessible if it had done more exploration. The agent may overlook arms with a greater genuine reward if it focuses too much on exploiting the arm with the highest perceived reward.

The "exploration" strategy, on the other hand, is picking weapons that aren't known to have the highest rewards in order to learn more about their prospective rewards. In order to learn more about the reward distribution and enhance its estimate of the expected benefit for each arm, the agent investigates the space of potential arms. The agent must use the exploration method to avoid becoming trapped in a poor arm and missing out on potential greater payouts. There are several algorithms created for multi-armed bandit issues that strike the correct balance between exploitation and exploration. Epsilon-greedy, which chooses the arm with the largest estimated payoff with probability (1 - epsilon) and a random arm with probability epsilon, is one of the most often used algorithms.

Overall, the key to achieving good performance in multi-armed bandit problems is to balance the exploitation and exploration strategies, using an algorithm that takes into account the uncertainty in the reward distribution and the agent's current estimate of the expected reward for each arm.

3. Explain action-value methods.

Action-value methods are a class of reinforcement learning algorithms that aim to learn the optimal action-selection policy for a given task by estimating the value of taking different actions in different states. The term "action-value" refers to the expected reward that can be obtained by taking a particular action from a particular state. By repeated interactions with an environment, an agent learns through reinforcement. The agent wants to discover a strategy that will maximise its long-term gain. Trial and error can be used to accomplish this, where the agent investigates its surroundings and modifies its policy in response to the rewards it receives.

Action-value methods calculate the value of performing a specific action in a specific state using a function called the action-value function, also referred to as the Q-function. The following is a definition of the action-value function:

$Q(s, a) = E[R \mid s, a]$

where s is the environment's current state, a denotes the agent's activity, R denotes the reward received by the agent, and $E[R \mid s, a]$ denotes the anticipated reward for the agent's action in state s.

The action-value function can be estimated using a variety of techniques, including Q-learning, SARSA, and Expected SARSA. The update rules and exploration tactics used by these algorithms vary, but they all strive to eventually converge on the best action-value function. After learning the action-value function, the agent can use it to select actions that maximise its expected long-term reward. This is typically accomplished through the use of an epsilon-greedy policy, in which the agent chooses the action with the highest estimated action-value with probability 1-epsilon and a random action with probability epsilon.

Overall, action-value methods are a strong and adaptable class of reinforcement learning algorithms that can be used to learn optimal policies in a variety of settings. However, they can be computationally expensive, especially when the state and action spaces are large, and good performance requires careful tuning of hyperparameters.